

停止写语气提示词：把品牌声音分层

SSS & Codex

2026 年 7 月 2 日



视频作者/频道：AI Engineer

发布日期：2026-06-26

视频时长：20:56

视频链接：<https://www.youtube.com/watch?v=ij-AU9dpJjc>

PDF 制作 Skill：<https://github.com/wdkns/wdkns-skills>

目录

1 为什么“语气提示词”会在第 21 轮失效	2
1.1 主结论：语气是产品边界，单层提示词承受不了四种责任	2
1.2 为什么模型更像“聪明实习生”	2
1.3 <i>happy path</i> 的成功会制造假安全感	3
1.4 四层控制栈：每一层只承担一种责任	4
1.5 固定顺序：从 24 个散落提示词到一个装配入口	5
1.6 Google Maps 类比：同一目的地需要不同路线	6
1.7 本章小结	7
2 Layer 1：不可变身份与品牌绝不能说的话	7
2.1 主结论：Layer 1 管的是“永远不能说”	7
2.2 AI 身份披露：透明是信任信号	8
2.3 身体在场边界：温暖不能制造虚假实体	9
2.4 缺失人口工具的例子：同一架构，不同伤害级别	9
2.5 从婚礼场地到失踪人员家庭：风险的共同结构	10
2.6 Layer 1 的工程落点	11
2.7 本章小结	11
3 Layer 2：情境模式把实时人类处境装进路线	11
3.1 第一组支持：先判断“车里坐的是谁”	11
3.2 第二组支持：软上下文备注给语气提供人类燃料	12
3.3 第三组支持：定性语气先于数字约束	13
3.4 例子：热度下降在不同语境里代表不同含义	14
3.5 本章小结	14
4 Layer 3：示例锚定语气只负责“好样子”	14
4.1 第一组支持：induction pack 把品牌声音具体化	15
4.2 第二组支持：示例覆盖的是已知路径	15
4.3 第三组支持：真实编辑能训练风格，仍然需要边界层	16
4.4 例子：好语气会放大错误事实的伤害	17
4.5 本章小结	17
5 Layer 4：生成后否决把“请求”变成“许可”	17
5.1 软标记：把可疑输出送去复核	18
5.2 硬拒绝：高成本具体事实必须对照现实	18
5.3 一个小公式：前三层影响概率，第四层给出放行判定	20
5.4 本章小结	21
6 多租户系统中的确定性边界与可改进之处	21
6.1 身份配置必须显式失败	21
6.2 请求与许可应由不同机制承担	22
6.3 收尾建议一：把 veto 做成共享服务	24
6.4 收尾建议二：集中条件解析	24

6.5	收尾建议三：正则拒绝与分类器的取舍	25
6.6	信任是系统边界	25
6.7	本章小结	26
7	总结与延伸	26
7.1	全片总论：品牌声音需要控制系统	26
7.2	讲者的收尾讨论：三个还应继续工程化的部位	26
7.3	给构建者的实践顺序	27
7.4	本章小结	27

1 为什么“语气提示词”会在第 21 轮失效

1.1 主结论：语气是产品边界，单层提示词承受不了四种责任

语气提示词会在第 21 轮失效，因为它把四件性质不同的工作压进同一个系统提示词：身份约束、实时情境、表达风格、输出审核。前几轮对话常常沿着 *happy path*（幸福路径）前进，团队提前写过例子，模型自然能模仿。第 21 轮对话（*turn 21*）代表第一个长尾问题：用户问法仍然合理，例子库已经覆盖不到，模型于是给出“技术上通顺、社会上灾难”的回答。

本章核心判断

品牌语气不能只写成一句“用我们的品牌声音回答”。在婚礼场地、豪华酒店、高端地产这类关系型业务里，*voice is the product*：用户买到的体验包含被怎样称呼、怎样安抚、怎样承诺、怎样被拒绝。语气一旦越界，损失会超过退款；它会直接破坏信任。

演讲者 Isadora Martin-Dye 的开场题目已经给出答案方向：停止写单层语气指令，把它拆成层。

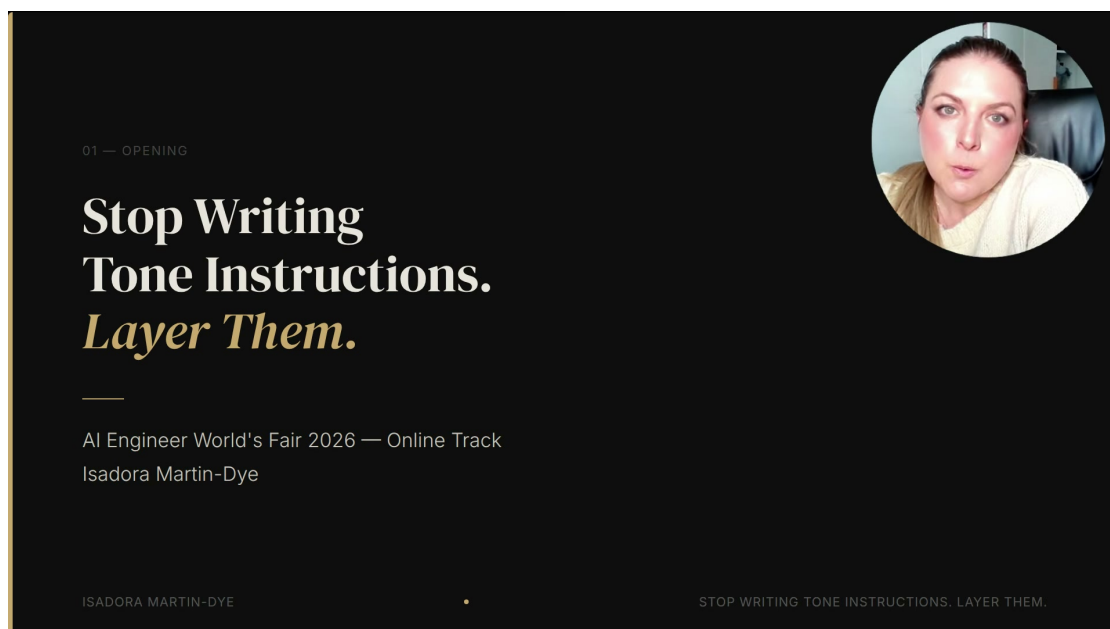


图 1：开场标题把全场问题压缩成一句话：*Stop Writing Tone Instructions. Layer Them.* 重点在“分层”，因为单个提示词无法稳定承担身份、情境、风格和审核四项职责。¹

1.2 为什么模型更像“聪明实习生”

演讲者给出的核心类比是 *brilliant intern*：一个高智商、低情商、记忆力极强的聪明实习生。这个类比很准确，因为大语言模型擅长记住前面给过的说明、复用语气样例、生成流畅文本；它缺少现场判断力，无法自然感知“这个句子在此刻会伤人”“这个承诺会被客户当真”“这个第一人称会让用户误以为对方有实体存在”。

¹ 视频来源区间：00:00:00.00–00:00:10.00。

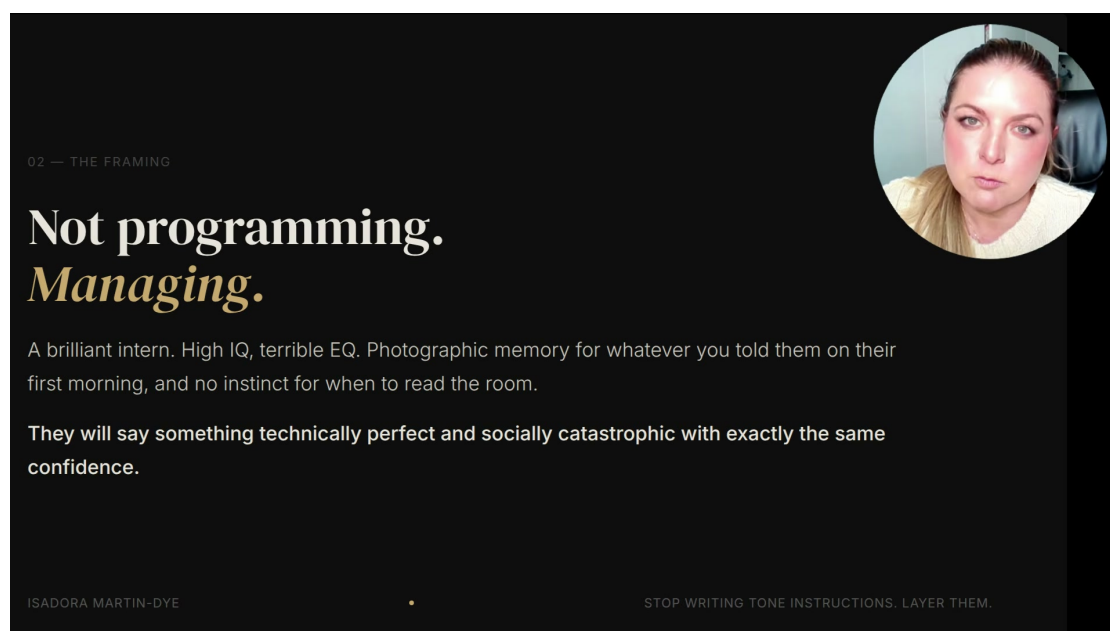


图 2: “Not programming. Managing.” 这张图给出管理模型的基本心智模型: 模型有高 IQ、低 EQ, 能说出技术上完美且社交上灾难的句子。²

类比的工程含义

把模型当作实习生, 会自然得到四个管理动作: 先讲公司底线, 再补充当前客户背景, 再给好例子, 最后在邮件发出前审一遍。对应到 AI 代理, 就是 Layer 1 到 Layer 4 的控制栈。这个类比把“写 prompt”从文案工作拉回到系统设计工作。

这也是“规则机器人”视角容易失手的地方。规则机器人只需写完规则就放行; 聪明实习生会把规则、例子和用户上下文揉在一起生成新句子。生成式系统的风险来自这个“揉合”动作: 输出常常像真人, 错误也常常像真人的自信承诺。

1.3 *happy path* 的成功会制造假安全感

happy path 指团队已经预料到的那部分问题: 常见咨询、常见拒绝、常见邀请、常见寒暄。模型在这里表现很好, 因为例子足够近, 模式足够稳定。问题在于这种成功会让团队误以为“语气指南已经解决了品牌声音”。第 21 轮对话才是真正测试: 用户的问题没有恶意, 场景却偏离了例子集。

²视频来源区间: 00:00:20.00–00:00:52.00。

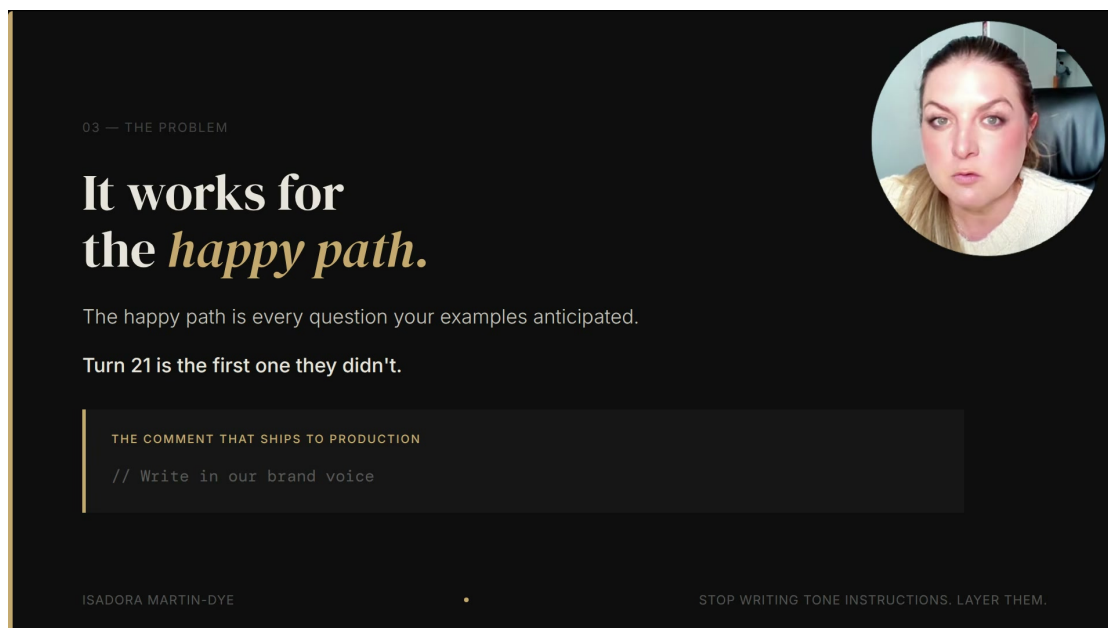


图 3: 这张问题页把失败机制讲得很直白: *happy path* 覆盖的是例子预料到的问题; *Turn 21* 是例子没有覆盖到的第一个问题。³

常见误区：把语气指南当作保证

“Write in our brand voice” 这类注释看起来像控制，其实只是在请求模型尽量模仿。它能改善常见表达，不能保证模型永远遵守身份边界、实时事实、禁用词、日期可用性和业务承诺。例子越漂亮，团队越容易忽视覆盖边界。

在零售搜索里，一个语气偏差可能只是体验瑕疵；在婚礼场地里，用户购买的是一段关系和仪式感。一个句子如果显得轻浮、虚假、越权，客户不会把它看成“模型小失误”，他们会把它看成品牌本身的判断力。

1.4 四层控制栈：每一层只承担一种责任

演讲者最终采用的方案是四层控制栈 (*four-layer stack*)。这四层按控制强度排序，越靠前越像硬边界，越靠后越接近输出审核。

³视频来源区间：00:01:00.00–00:01:32.59。

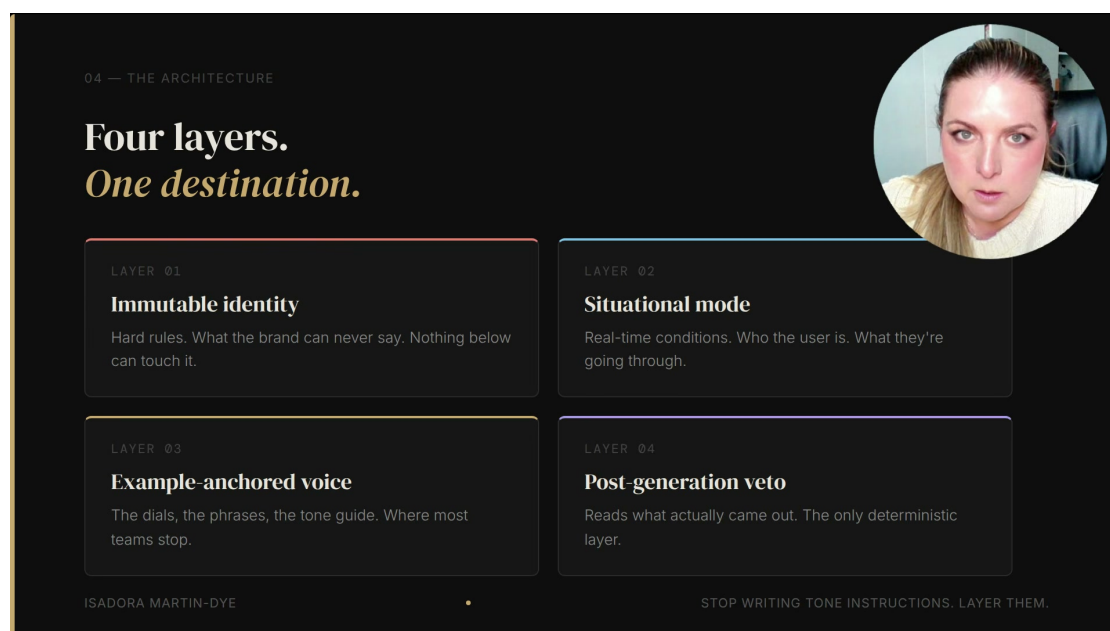


图 4: 四层总览: Layer 1 是 *immutable identity*, Layer 2 是 *situational mode*, Layer 3 是 *example-anchored voice*, Layer 4 是 *post-generation veto*。⁴

四层各自解决的失败模式

Layer 1 (*immutable identity*, 不可变身份层) 规定品牌永远不能说什么。Layer 2 (*situational mode*, 情境模式层) 注入当前用户是谁、正在经历什么、实时条件如何。Layer 3 (*example-anchored voice*, 示例锚定语气层) 提供语气样例、短语和风格偏好。Layer 4 (*post-generation veto*, 生成后否决层) 读取已经生成的草稿, 决定它能否离开业务系统。

这四层的关键价值在于分工清楚。身份层处理“绝不能”; 情境层处理“此刻怎样读懂人”; 示例层处理“听起来像谁”; 否决层处理“已经写出来的内容能否放行”。把这四项任务写进同一段提示词, 模型会在常规场景中看起来稳定, 在边界场景中暴露真实脆弱性。

1.5 固定顺序: 从 24 个散落提示词到一个装配入口

演讲者提到, 旧系统里曾有 24 个系统提示词散落在代码库中; 有的叫 Sage, 有的叫 Venue, 不同表面各自理解“我是谁”。这种结构会制造隐形分叉: 一个产品看起来只有一个 AI 代理, 实际每条路径都可能带着不同身份、不同语气和不同约束。

⁴视频来源区间: 00:01:32.59–00:03:02.59。

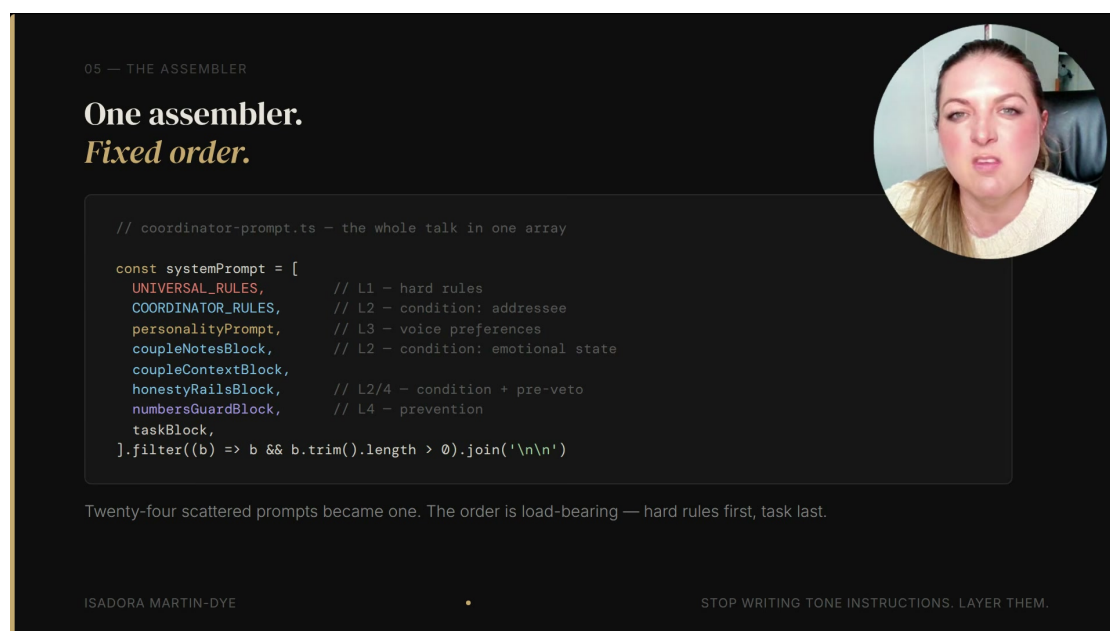


图 5: 装配器代码页显示一个固定顺序的 `systemPrompt` 数组: `UNIVERSAL_RULES` 在前, 任务块在后, 中间依次加入协调员规则、人格、用户备注、诚实性检查和数字守门。⁵

这里的非显而易见选择是“为什么要一个装配入口, 并且固定顺序?” 答案很硬: 品牌身份和业务底线必须先进入上下文, 具体任务必须后进入上下文。若任务先把模型推向一个热情路线, 后面再补规则, 就像导航已经拐错弯以后才开始提醒施工封路。顺序本身就是控制面。

“装配入口”是什么

装配入口可以理解为所有回答生成前的统一检查站。它把通用规则、场景规则、人格配置、客户上下文、事实约束和任务请求按确定顺序组合起来。好处是每个产品表面都经过同一条入口, 团队可以审计“回答到底带了哪些控制信息”。

1.6 Google Maps 类比: 同一目的地需要不同路线

Google Maps 类比把层级顺序讲得很清楚。目的地是“给出正确回答”; 交通规则对应 Layer 1; 实时路况、修路、油量和驾驶者状态对应 Layer 2; 偏好路线和停靠点对应 Layer 3; 出发前检查路线对应 Layer 4。导航系统只有在拿到规则和实时条件之后, 才有资格规划路线。

⁵ 视频来源区间: 00:03:02.59–00:04:32.59。

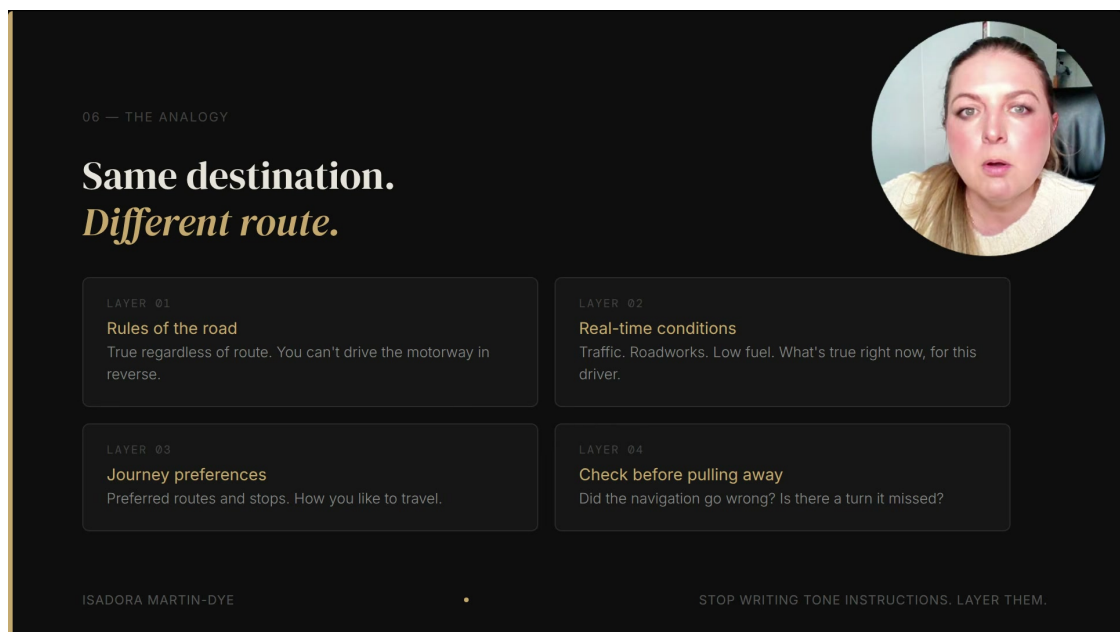


图 6: 路线类比把四层映射为交通规则、实时条件、旅程偏好和出发前检查。它说明同一目标可以有不同路线，前提是约束和条件进入系统的顺序正确。⁶

这个类比对构建 AI 代理特别有用：语气不能当作最后撒上的调味料，它要在正确约束内工作。婚礼场地可以温暖、体贴、有仪式感；这些风格偏好只有在身份披露、身体在场边界、实时事实和输出审核都成立时才安全。第 21 轮失败的根因通常是系统把“旅程偏好”当成了整套导航。

1.7 本章小结

本章建立了全片的主问题：单层语气提示词会在长尾对话里失效，因为它把品牌身份、当前情境、表达样式和输出放行混成一件事。演讲者用 *brilliant intern* 解释模型的高能力与低判断力，用 *happy path* 和 *turn 21* 解释例子覆盖的边界，用四层控制栈和 Google Maps 类比说明正确的工程解法。

读者应带走三点。第一，关系型业务里的 AI 语气是一条信任边界。第二，示例能提高常规表现，无法覆盖所有高风险边界。第三，可靠的 AI 代理需要分层治理：先固定不可违反的身份规则，再注入实时情境，再提供语气样例，最后检查生成结果能否放行。

2 Layer 1：不可变身份与品牌绝不能说的话

2.1 主结论：Layer 1 管的是“永远不能说”

Layer 1 是 *immutable identity*（不可变身份层）：它规定 AI 代理在任何场景、任何租户、任何语气配置、任何用户诱导下都不能越过的身份边界。它的性质高于“风格偏好”和“尽量遵守的建议”，属于品牌结构的一部分。路线可以变化，规则不会跟着变化。

⁶ 视频来源区间：00:03:02.59–00:04:32.59。

Layer 1 的定义

不可变身份层回答一个问题：这个 AI 代理代表业务开口时，哪些话永远不能说？典型内容包括 AI 身份披露、禁止假装真人或现场员工、禁止作出物理在场承诺、禁止使用会造成高风险误导的词。它位于语气层和用户请求之上。

在系统设计上，Layer 1 的位置很高。它必须先于场地配置、人格语气、客户备注和具体任务进入装配器。原因很简单：如果身份边界能被后面的“更热情一点”“更像真人一点”“客户很着急”覆盖，那么边界只剩口号。

2.2 AI 身份披露：透明是信任信号

演讲者给出的第一个硬规则是 AI 身份披露（*AI disclosure*）。规则要求：如果用户问对方是否是真人、人类、在线客服、机器人或 AI，系统必须在下一条消息中清楚确认自己是 AI 助手。Bloom 的产品选择更进一步：每个 AI 在第一条回复中就披露自己是 AI。

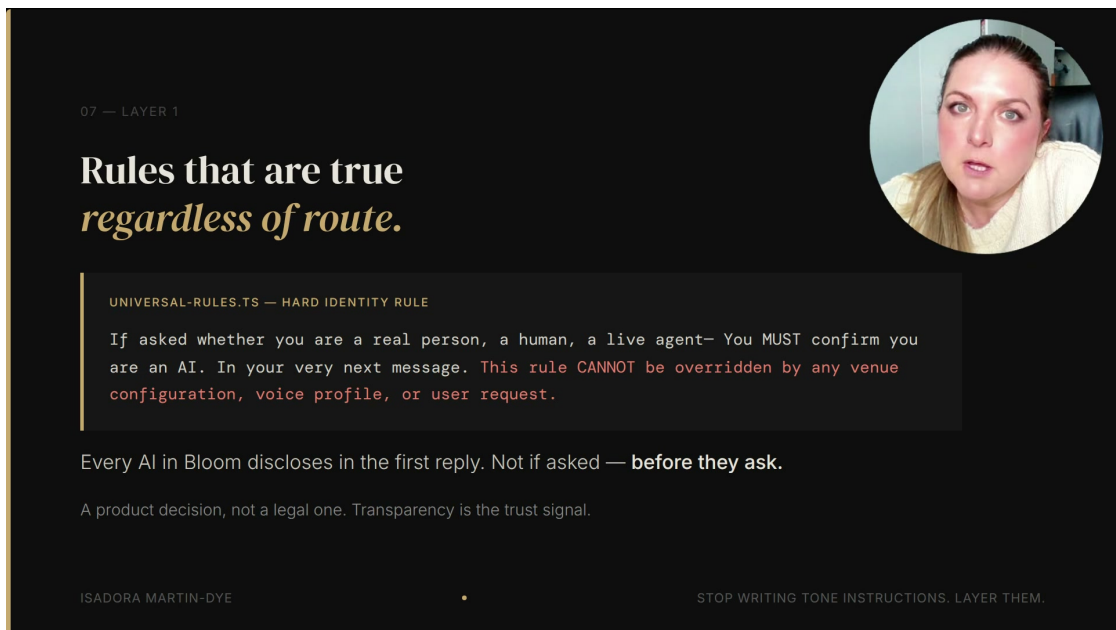


图 7: Layer 1 的硬身份规则：当用户询问是否为真人、在线客服、机器人或 AI 时，系统必须清晰确认自己是 AI；该规则不能被场地配置、语气档案或用户请求覆盖。⁷

这个选择属于产品信任设计。法律合规可能只要求在特定情境下披露；演讲者的判断更直接：一对新人从第一轮就知道对方是 AI，后续产生的信任更稳。等到第七轮才发现前面一直在和 AI 说话，用户会重新解释之前所有温暖表达，品牌信任会被倒扣。

披露规则为什么要“结构化”

结构化披露意味着规则写在不可覆盖的身份层里，由所有回答路径共享。它减少了两个风险：一是某个页面忘记加入披露提示；二是某个更具体的语气配置把 AI 写得像真人。对用户来说，透明是判断关系边界的前提，礼貌表达只能排在它后面。

⁷ 视频来源区间：00:04:32.59–00:06:02.59。

2.3 身体在场边界：温暖不能制造虚假实体

第二个硬规则是身体在场边界 (*physical presence boundary*)。AI 是软件，没有身体，不能带客户看房，不能在婚礼场地亲自接待，不能说“我迫不及待想见到你”。这些句子在真人员工那里很温暖；由 AI 说出口，就会让温暖变成事实性误导。

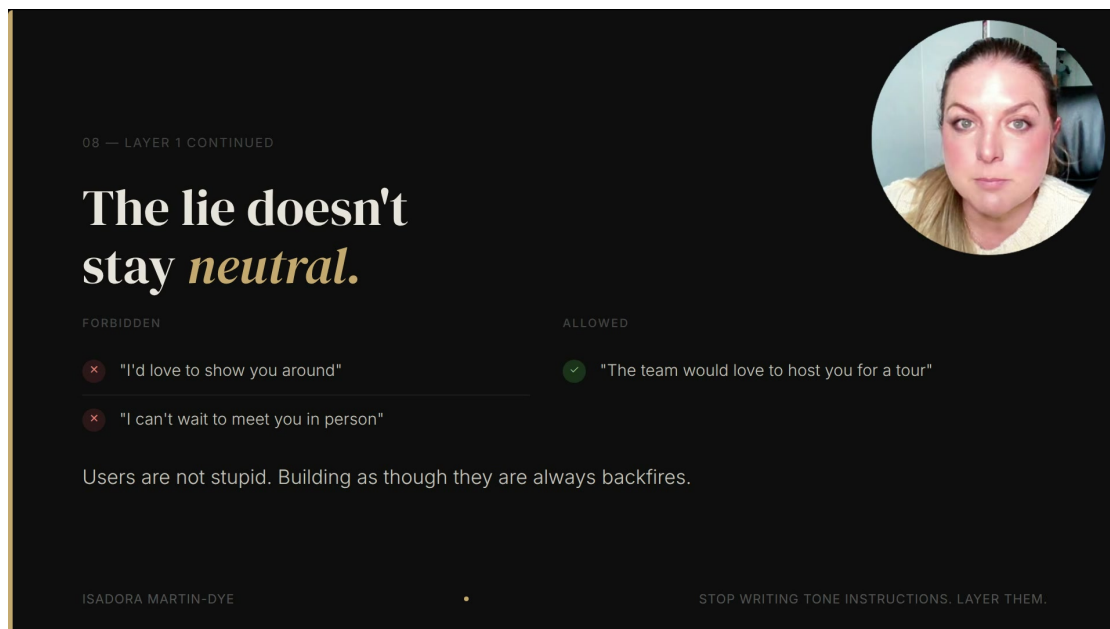


图 8：身体在场边界的可视化规则：禁止说“I’d love to show you around”或“I can’t wait to meet you in person”；允许的表达式是“The team would love to host you for a tour”。⁸

这里的关键是句子制造了什么关系预期，句子是否好听排在后面。用户听到“I can’t wait to meet you in person”时，会把“我”理解成某个即将出现的人。后来发现“我”从来没有身体，信任不会轻微下降，它会反向翻转：用户会怀疑自己刚才是在对一个伪装成人的系统投入情绪。

常见失败点：第一人称热情会暗示身体

很多品牌语气指南鼓励第一人称，因为第一人称显得亲近。AI 代理使用第一人称时，要先检查它是否暗示了实体能力。可用表达通常把动作交还给真人团队，例如“团队很期待接待你参观”；风险表达则让软件承担身体动作，例如“我带你参观”。

这条规则尤其适合用 Layer 1 实现，因为下层语气会天然追求温暖。若只让 Layer 3 学“亲切、欢迎、兴奋”，模型很容易把真人员工的邀请句式复制过来。Layer 1 的作用就是让语气层在不可越过的现实边界内发挥。

2.4 缺失人口工具的例子：同一架构，不同伤害级别

演讲者的跨产品例子更尖锐。画面中的产品名写作 Threadline；转写稿中可能出现 Thread Light。它面向失踪人员家庭，声音风格与婚礼场地完全不同，架构仍然相同。Layer 1 在这里承载一条更重的规则：禁止使用 *confirmed*、*identified*、*matched*、*proven*、*linked*、*solved* 这类词。

⁸ 视频来源区间：00:06:02.59–00:07:32.59。

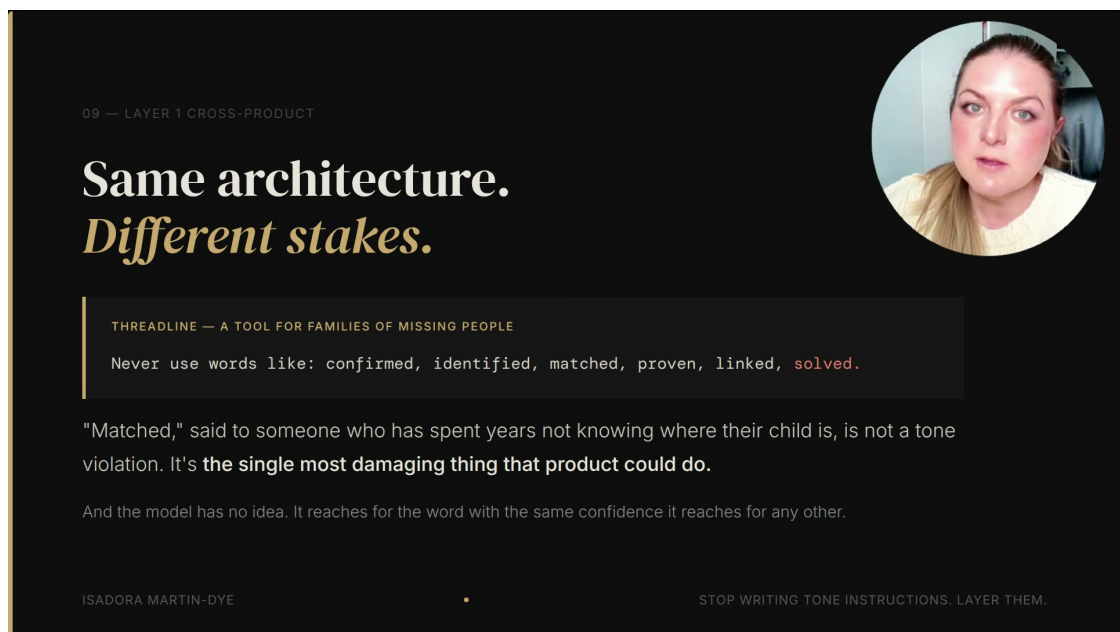


图 9: 缺失人口场景把 Layer 1 的风险级别拉高: 同一套架构服务不同产品, 禁用词从品牌体面问题升级为可能伤害家庭的高风险表达。⁹

这些词在一般数据产品里很自然。模型看到相似线索时, 统计上会倾向使用“matched”或“linked”。问题在于, 向一个多年寻找亲人的家庭说出“matched”, 会被理解成命运级别的消息。模型没有悲悯判断力, 它只是选择高概率词。Layer 1 必须把这些词从可选空间里拿掉。

为什么禁用词属于身份层

禁用词规则表达的是产品身份: 这个系统没有资格宣布某个人被确认、识别、匹配、证明、关联或解决。这里处理的是权限边界, 措辞优化只能服从这个边界。模型越自信, 越需要更高层的硬约束限制它能说的话。

2.5 从婚礼场地到失踪人员家庭: 风险的共同结构

婚礼场地和缺失人口工具看起来相距很远, 底层风险结构一致: AI 代理用品牌身份说话, 用户会把它的话当成关系承诺或事实信号。差别在于伤害强度。婚礼场地里, AI 假装有身体会造成尴尬和信任翻转; 缺失人口场景里, AI 使用“确认”“匹配”这类词, 可能让家庭承受灾难性的情绪冲击。

互补概念: 语气层与身份层

语气层负责“怎样听起来像品牌”, 身份层负责“品牌拥有怎样的说话资格”。二者互补: 没有语气层, 回答会僵硬; 没有身份层, 温暖会越权。强系统先界定资格, 再塑造表达。

这解释了为什么 Layer 1 不能被用户指令覆盖。用户可以要求“说得更亲密一些”, 场地可以要求“更像真人管家”, 营销团队可以要求“更有情绪温度”。这些请求都不能让 AI 获得身体, 也不能让系统获得宣布失踪人员结果的资格。

⁹视频来源区间: 00:07:32.59-00:09:02.59。

2.6 Layer 1 的工程落点

不可变身份层应当放在统一装配器的最前面，并且所有产品表面都要经过它。工程上可以把它理解成“上游约束”：一旦系统要生成对外文字，先加载硬身份规则，再加载场景和语气。这样做能避免三个常见事故：

1. 某个新页面忘记声明 AI 身份，导致用户在多轮对话后才发现对方是 AI。
2. 某个语气档案追求亲切，诱导模型说出“我会亲自接待你”之类的身体承诺。
3. 某个高风险产品复用通用模型词汇，把统计上自然的词放进情绪上不可承受的场景。

读者容易忽视的未知风险

Layer 1 的难点在于“哪些话永远不能说”通常不会从普通功能需求里自然长出来。团队需要主动询问：哪些词会被用户当成承诺？哪些第一人称会暗示真人？哪些产品场景会让普通词汇变成高风险信号？缺少这一步，品牌语气会在最敏感的地方替系统越权。

2.7 本章小结

Layer 1 是四层控制栈里的底线层。它负责 AI 身份披露、身体在场边界、禁用高风险词和不可覆盖的品牌资格。婚礼场地例子说明温暖表达会暗示虚假实体；Threadline 缺失人口例子说明普通词汇在高情绪场景里会造成严重伤害。

本章的实践结论很直白：语气层可以让系统更像品牌，身份层决定品牌有资格说什么。凡是会制造虚假信任、实体误导、事实承诺或情绪灾难的话，都应该进入不可变身份层，并且位于所有场地配置、用户请求和风格样例之上。

3 Layer 2：情境模式把实时人类处境装进路线

Layer 2 的主张很直接：情境模式层（situational mode）要在模型生成之前，把实时条件（real-time conditions）写进路线。Layer 1 规定 AI 代理永远不能跨越哪些边界；Layer 2 负责回答另一个问题：此刻和它对话的人是谁，他们正处在什么状态，当前事实会怎样改变一条合适回应的路径。

Layer 2 的定义

Layer 2 是动态路由层。它保留同一套身份规则和同一个服务目标，同时根据角色、人生处境、实时条件和软上下文备注（soft context notes）改变回应路线。它的任务是让 AI 代理在开口之前先读懂场景。

这层的价值来自一个朴素事实：同一句话放到不同人面前，风险和效果会完全改变。面向内部协调员时，AI 可以给出趋势、置信度、异常提示和运营判断；面向新人客户时，同样的信息需要转成服务型语言、时间感和照顾感。Google Maps 类比在这里很有用：目的地没有变，路况、乘客、燃油、施工和偏好会改变路线。

3.1 第一组支持：先判断“车里坐的是谁”

视频在 00:09:02.59–00:10:32.59 先讲角色上下文。讲者强调，同一个 AI 既会和新人客户对话，也会给场地工作人员做 briefing。身份相同，语气系统相同，服务目标也相同；一旦受众改变，路线

就要改变。

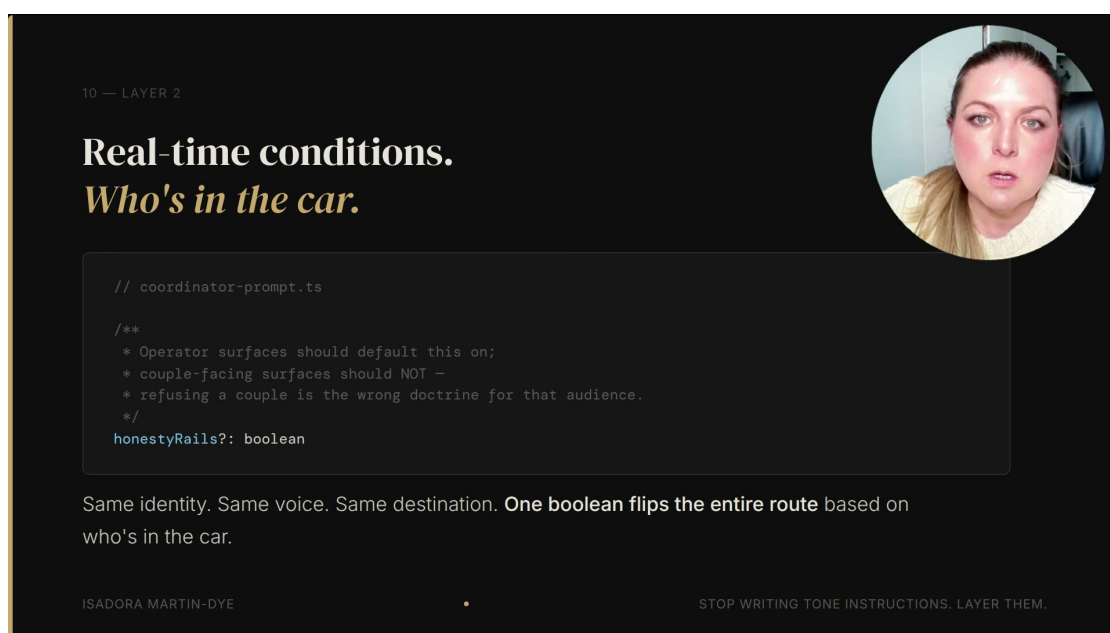


图 10: Layer 2 的第一类实时条件是 “Who’s in the car”: 同一身份、同一语气、同一目的地，会因为受众不同而切换路线。¹⁰

画面中的 `honestyRails?: boolean` 很小，却代表一个很大的产品决策：内部运营界面可以默认打开更直接的诚实轨道，让系统说“这个预测没有足够把握，这里是趋势，你可能会看到这些变化”。新人客户界面需要另一种路线，重点变成安抚、解释和后续服务动作。

受众改变路线

“谁在车里”指向的范围超过称谓切换。它决定信息密度、风险暴露、语气力度和行动建议。内部同事能承受概率语言、风险提醒和运营假设；客户通常需要清楚、温和、可执行的回应。

这个点容易被低估。很多团队会把角色差异写成模板变量，例如把“用户”替换成“协调员”或“新人”。Layer 2 更深一层：它改变的是回答策略。协调员需要知道趋势能否被预测；新人需要知道下一步怎么安心推进。若把两种路线混在一个系统提示词里，模型会在长对话中随机抽取一种风格，结果要么过度内部化，要么过度客服化。

3.2 第二组支持：软上下文备注给语气提供人类燃料

第二类实时信号来自用户正在经历的事情。讲者把它称为 `soft context notes`：这些备注用于 `tone`、`empathy` 和 `what not to say`。这里的关键动词是“用于”，系统要吸收它们来调整表达方式，而不能把它们原样交还给用户。

例如，正在处理丧亲压力的新人应该听到更慢、更轻、更少催促的语气；父母正在化疗的客户应该获得时间上的宽容和安排上的弹性。系统不应把这些备注写成“因为你的家人正在生病，所以……”。那样会把后台敏感事实变成前台话术，既突兀，也容易伤害信任。

¹⁰ 视频画面时间区间：00:09:02.59–00:10:32.59。

软上下文使用边界

软上下文备注的第一条安全规则是禁止逐字引用。它们像驾驶员看到的路况提醒：驾驶员据此减速、绕行、提前预留时间，却不会对乘客机械播报每一条内部备注。

这也是为什么“情境模式”比普通个性化更难。个性化经常只是记住偏好；Layer 2 要处理用户处境。偏好可以直接说出来，处境经常需要被温柔地吸收。对 AI 代理而言，真正困难的是在不暴露敏感备注的前提下，让回应显得有人味。

3.3 第三组支持：定性语气先于数字约束

视频里最值得工程团队记住的细节，是 assembler 的渲染顺序：couple notes 要在 numbers guard 之前进入提示。讲者给出的理由很具体：模型先读到人类处境，才会用合适的语气满足后面的数字约束；若它先进入数字框架，后面补进来的语气材料会显得像硬塞进去的补丁。

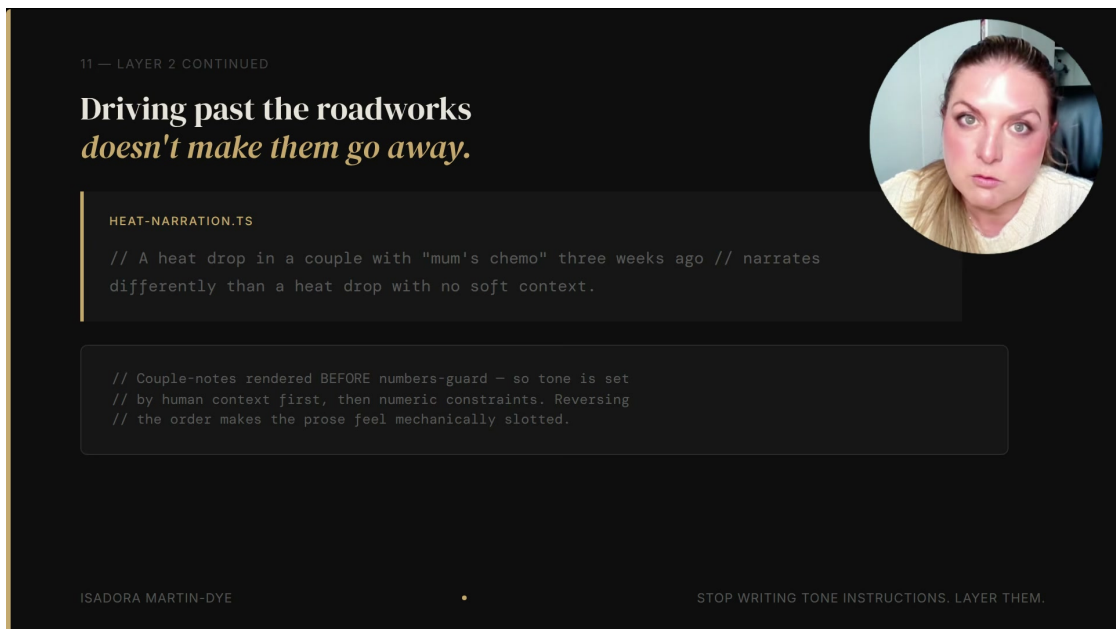


图 11: “Driving past the roadworks doesn’t make them go away.” 软上下文要先塑造语气，再让模型满足数字约束。¹¹

这段的隐含工程原则是：上下文顺序会改变模型的注意力锚点。数字约束通常很强，例如日期、次数、价格、可用性、热度指标。它们一旦先出现，模型容易把回答组织成“指标解释”。人类处境先出现时，模型会把同样的指标解释放进更合适的关系语境里。

先放人，再放数

Layer 2 的顺序规则：先放人，再放数。先让模型知道这个用户正在经历什么，再要求它满足日期、热度、可用性、价格等具体约束。这样做能降低机械感，也能减少把人当成指标的风险。

这里有一个“未知的未知”：实时上下文不只提升体验，也会引入隐私和治理风险。团队需要明

¹¹ 视频画面时间区间：00:10:32.59–00:12:02.59。

确定哪些备注可以进入提示，哪些备注只允许影响路由，哪些备注需要过期，哪些备注只能由人工维护。Layer 2 的价值越大，数据边界越需要严肃设计。

3.4 例子：热度下降在不同语境里代表不同含义

讲者给出的最佳例子来自 heat map：系统会记录新人客户联系频率的变化。如果某对新人近期互动降低，纯数字读法可能把它当成冷线索、流失风险或需要追催的客户。若系统知道客户的母亲过去三周在化疗，同一个热度下降就应被读成家庭承压信号。

这里的问题已经超出语气润色。真正改变的是解释框架。一个没有 soft context 的热度下降，可以触发运营追踪；带有家庭医疗背景的热度下降，应触发耐心、空间、少催促、明确但轻柔的下一步。AI 代理的语气没有换品牌，换的是对信号的解读。

读数解释器

可以把 Layer 2 想成“读数解释器”。仪表盘只告诉系统某个指标变了；情境模式告诉系统这个变化在人类生活里可能意味着什么。没有这层，AI 容易把所有下降都读成问题，把所有沉默都读成不配合。

这也解释了为什么讲者说“driving past the roadworks”无法让 roadworks 消失。系统假装没有施工，路线只会更差；AI 代理假装没有用户处境，回应只会更硬。人类处境本来就在路上，Layer 2 的任务是让它进入路线规划。

3.5 本章小结

Layer 2 把“实时人类处境”变成生成前的路线条件。它主要处理三组信号：第一，受众是谁，内部协调员和新人客户需要不同路线；第二，用户正在经历什么，软上下文备注要转化为语气和禁区；第三，提示装配顺序如何影响模型先读人还是先读数。

本章最重要的 takeaway 是：情境模式层让 AI 代理诚实地使用它已经知道的上下文。Layer 1 防止 AI 编造身份和能力；Layer 2 防止 AI 把真实的人类处境当成无关噪声。下一章进入 Layer 3：示例、短语、语气刻度和人工编辑样本怎样教模型“好样子”，以及它们为什么只能负责风格。

4 Layer 3：示例锚定语气只负责“好样子”

Layer 3 的主张也很清楚：示例锚定语气层（example-anchored voice）负责教模型“好回答长什么样”，它适合承载 tone guide、phrase list、style dials、approved phrases、banned phrases、USP 和人工编辑样本。它能显著改善品牌声音，却只能负责风格学习，无法承担身份硬约束、实时情境路由和生成后审查。

Layer 3 的定义

Layer 3 是风格训练层。它把抽象的“写得像我们”压成可学习的例子、短语、刻度和编辑记录。它的核心产出是“what good looks like”，也就是模型在已知场景里应当模仿的好样子。

这层最容易让团队产生安全感，因为它看起来最像传统品牌工作：市场团队给出语气指南，工程师把它接入 prompt，系统马上更像品牌在说话。视频里的提醒很硬：大多数团队会在这里停下，因为 Layer 3 的成果最可见，也最容易被误认为已经覆盖了整个问题。

4.1 第一组支持：induction pack 把品牌声音具体化

讲者沿用前文的聪明实习生类比，把 Layer 3 称作 induction pack：第一天交给实习生的入职资料包。它里面有好例子、常用表达、禁用表达、语气刻度和场地卖点。实习生读完以后，确实更容易写出像这个品牌的句子。

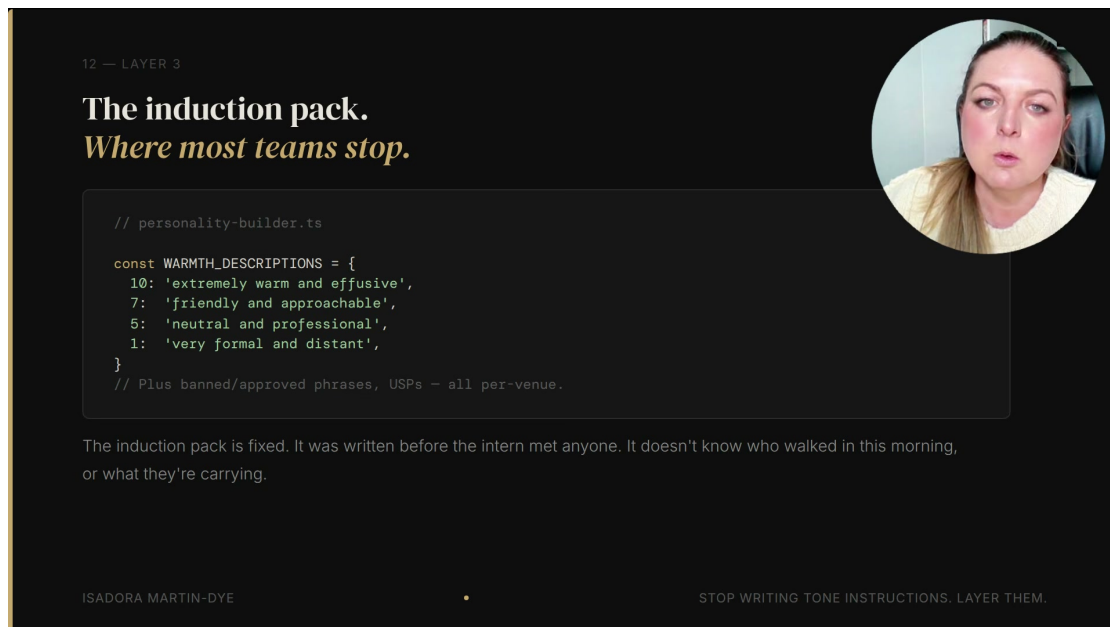


图 12: Layer 3 的 induction pack: `personality-builder.ts` 用 warmth scale 和每个 venue 的短语材料，把抽象语气变成可学习配置。¹²

画面中的 warmth scale 很适合解释 Layer 3 的有效性：10 可以是 extremely warm and effusive，7 可以是 friendly and approachable，5 是 neutral and professional，1 是 very formal and distant。这样的刻度把模糊的品牌感受转换成模型更容易使用的条件。短语列表和 USP 则提供词汇、节奏和卖点边界。

风格样本库

Layer 3 像给模型建立“风格样本库”。样本越贴近真实业务，模型越容易学会句子的长度、礼貌程度、称呼方式、情绪密度和品牌偏好的词。它提高的是表达质量和一致性。

这层的正面价值应被承认。没有 Layer 3，AI 代理容易落回通用客服腔：礼貌、流畅、缺少品牌细节。对婚礼场地、精品酒店、高端地产这类“voice is the product”的业务来说，通用腔本身就会损害体验。示例锚定语气可以把品牌从抽象价值观拉回可复用句式。

4.2 第二组支持：示例覆盖的是已知路径

induction pack 的边界同样来自它的定义：它在实习生见到当天第一个真实客户之前就已经写好。它可以告诉模型过去哪些回答好，却无法预知今天走进来的每一个人、每一种处境、每一次异常请求。

因此，示例最擅长覆盖 happy path，也就是团队已经预料到并提供过样本的场景。到了 turn 21，用户提出样本从未覆盖的情况，phrase list 就没有明确答案。模型仍会努力模仿语气，风险恰

¹² 视频画面时间区间：00:12:02.59–00:13:32.59。

好在这里升高：句子听起来越像品牌，用户越容易相信它。

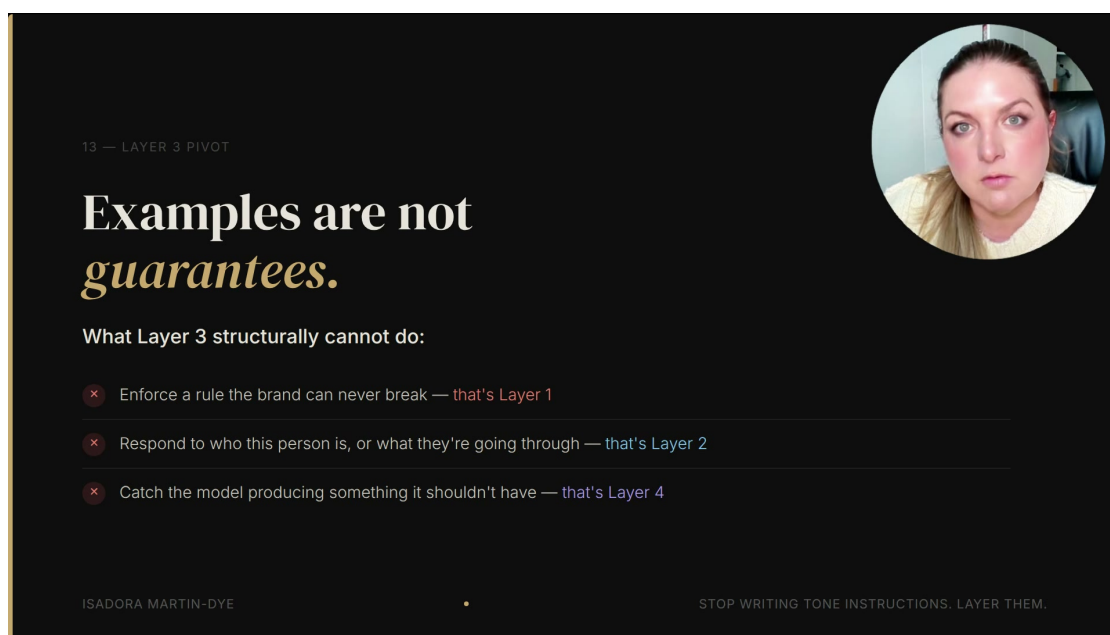


图 13: “Examples are not guarantees.” 示例能教好样子，无法独自承担 Layer 1、Layer 2 和 Layer 4 的工作。¹³

这张图把 Layer 3 的边界压缩得很清楚：它不能强制执行品牌永远不能违反的规则，那是 Layer 1；它不能根据此刻这个人的身份和处境重选路线，那是 Layer 2；它不能读取生成后的文本并拦下危险输出，那是 Layer 4。

示例不是保证

把示例当成保证，是 Layer 3 最常见的误用。示例会让模型更像品牌说话；保证需要更强的控制面。身份边界、处境路由、事实校验和输出否决，都不能交给 phrase list 独自承担。

4.3 第三组支持：真实编辑能训练风格，仍然需要边界层

视频里还提到 Bloom 的 voice training exercise：系统可以从协调员对 AI 回答的真实编辑中学习。这个机制很有价值，因为它比静态 tone guide 更贴近业务现场。协调员反复把某类句子改短、改暖、改具体，模型就能学到品牌在真实场景里的偏好。

这种反馈环节可以理解为“把人工品味变成样本”。它减少抽象指令，增加可模仿证据。对工程团队来说，这比写一句“be warm, elegant, and concise”更可靠，因为模型看到的是被人类确认过的成品。

人工编辑样本

人工编辑样本的价值在于降低风格歧义。它告诉模型：同一个事实可以怎样更体面地说，哪些词太生硬，哪些承诺太满，哪些句子更像这个品牌的服务习惯。

边界依然清楚。人工编辑样本可以让句子更自然，无法让 AI 拥有身体，无法判断客户家庭处境，无法核对日历日期。把这些职责继续叠给 Layer 3，会让团队得到一种危险的错觉：回答越来越

¹³ 视频画面时间区间：00:12:02.59–00:13:32.59。

好听，系统控制却没有相应增强。

4.4 例子：好语气会放大错误事实的伤害

Layer 3 的真正风险不在“写得差”，反而在“写得太像”。如果模型用非常温暖、非常符合品牌的口吻承诺了不存在的日期、错误的价格或越界的能力，用户受到的影响会更强。冷冰冰的错误容易被怀疑；高品质品牌语气包装过的错误更像真实服务承诺。

这就是下一章 Layer 4 的入口。Layer 3 可以教模型怎样表达善意；Layer 4 要决定这段善意能否离开系统。前者是风格，后者是许可。把两者分开，团队才能既保留品牌温度，又避免温暖语气替错误事实增加可信度。

Layer 3 的停止点

Layer 3 的合格标准是“像品牌”，它的停止点也是“像品牌”。当问题进入事实、身份、政策、日期、价格、承诺、隐私和高风险措辞，系统需要其他层接管。

4.5 本章小结

Layer 3 是必要的风格层：它用 induction pack、warmth scale、短语列表、场地卖点和协调员编辑样本，让 AI 代理学会品牌的好样子。它对 happy path 特别有效，也能显著减少通用客服腔。

本章最重要的 takeaway 是：示例负责表达质量，不能提供系统保证。Layer 1 管身份硬边界，Layer 2 管实时人类处境，Layer 4 管生成后的许可。Layer 3 放在中间，承担它最擅长的工作：让正确路线上的回答听起来像这个品牌。

5 Layer 4：生成后否决把“请求”变成“许可”

本章主张

Layer 4 的核心价值在于把模型已经写出的草稿交给一个生成后的检查者：前三层告诉模型应该怎样写，Layer 4 决定这段文字能否离开系统。原讲者把这一层称为 **post-generation veto (生成后否决)**，它的关键句是：“**Instructions are probabilistic. Permission is deterministic.**”

前三层都发生在生成之前：Layer 1 提供不可变身份，Layer 2 注入当前处境，Layer 3 给出示例锚定语气。它们能提高好输出出现的概率，却无法保证最终文本真的安全、真实、合规。Layer 4 的位置完全不同：它读取 y ，也就是模型已经吐出的草稿，并把结果分成允许、标记、拒绝三类。对业务系统来说，这一步把“请你遵守”升级成“通过检查才放行”。

14 — LAYER 4

The only layer that reads what *actually* came out.



```

// honesty-rails.ts - soft flag (regex, microseconds)

if (FORECAST_QUESTION_PATTERNS.test(question) &&
    !FORECAST_HEDGE_PATTERNS.test(response)) {
  flags.push({
    rule: 'forecast_no_hedge',
    reason: 'Response did not hedge or name confounds.'
  })
}
// Conservative by design: false positives are cheap,
// false negatives are the real cost.

```

A false positive: someone double-checks a fine response. A false negative: a confident invention ships to a couple who now believes it.

ISADORA MARTIN-DYE

STOP WRITING TONE INSTRUCTIONS. LAYER THEM.

图 14: Layer 4 是唯一读取实际输出的层：示例中的 `honesty-rails.ts` 用软标记检查预测类回答是否缺少必要的 hedging。¹⁴

5.1 软标记：把可疑输出送去复核

第一类 veto 是 **soft flag**（软标记）。它不一定阻止回复发送，主要职责是把可疑文本放入人工或二次系统复核队列。讲者举的例子是 **honesty inspector**（诚实性检查器）：如果用户问的是预测类问题，回答却没有给出不确定性边界，也没有说明混杂因素，检查器就把它标出来。

误报与漏报的成本不对称

false positive（误报）的成本通常是多一次复核；**false negative**（漏报）的成本可能是一个编造数字、隐私问题、错误政策或虚假承诺被客户看到。工程上应优先控制漏报，因为漏报会把系统内部的不确定性外包给用户承担。

这个不对称性解释了为什么生成后检查值得单独成层。很多团队把“模型是否真的回答了问题”视为提示词质量问题，实际系统里它更像质量闸门：可疑答案需要被拦下来、打标签、记录原因，并进入可追踪的复核路径。

5.2 硬拒绝：高成本具体事实必须对照现实

第二类 veto 是 **hard reject**（硬拒绝）。它处理讲者所谓的昂贵失败：模型自信地说出没有依据的数字、日期、价格、政策或承诺。这里的 **numbers guard**（数字守门器）应理解为一个事实守门组件，它将输出里的具体值与授权来源核对；不在允许列表里的值无法发送。

¹⁴ 视频画面时间区间：00:13:32.59–00:15:02.59。

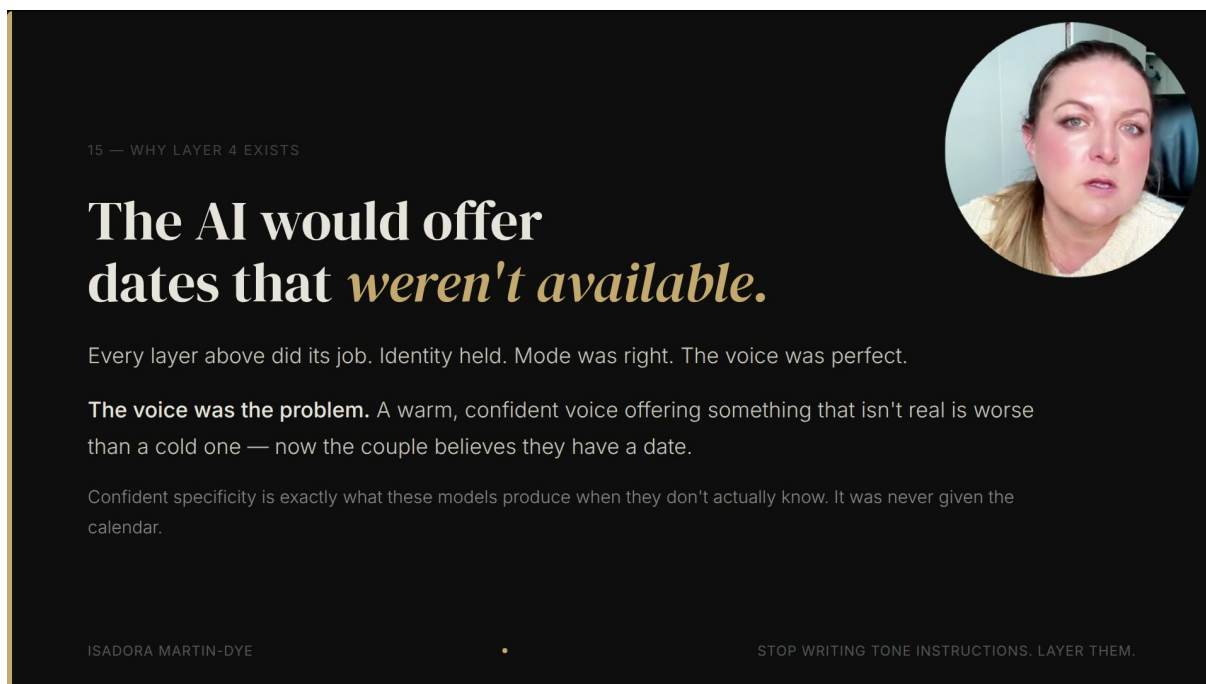


图 15: Layer 4 出现的直接原因: AI 会用温暖、自信的语气提供实际上不可用的日期, 前三层都表现正常, 错误仍然会发生。¹⁵

婚礼场地的日期例子最能说明问题。情侣询问十月某个周六, 模型想表现得温暖、专业、具体, 于是写出“很愿意为你们保留这个日期”之类的话。若日历里这个日期已经被订走, 这句话会制造一个延迟爆炸的承诺: 用户相信自己得到了一个可用日期, 48 小时后才发现它从未存在。

温暖会放大错误事实的伤害

冷冰冰的错误已经糟糕; 带有品牌亲密感的错误更危险, 因为它让用户把虚假事实当成服务承诺。日期、价格、政策、身份、隐私和法律相近陈述都应接入确定性核验来源。

¹⁵ 视频画面时间区间: 00:13:32.59–00:15:02.59。

Check the specifics *against what's real.*

```
// heat-narration.ts - numbers-guard hard reject

if (!result.ok) {
  if (result.numbersGuardViolations) {
    console.warn(
      '[heat-narration] numbers-guard rejected:',
      result.numbersGuardViolations.map(v => v.token).join(', ')
    )
  }
  // Degrade gracefully - don't cache. Re-attempt next run.
  return { ...narration, confidence: conf.value, cached: false }
}
```

A date the model offered that isn't in the allowlist doesn't ship. Prevention is the prompt. The veto is the check. You need both.

ISADORA MARTIN-DYE

STOP WRITING TONE INSTRUCTIONS. LAYER THEM.



图 16: 硬拒绝的工程形态: `numbers-guard` 发现违反允许列表的具体值后拒绝缓存与发送, 下一轮重新尝试。¹⁶

5.3 一个小公式: 前三层影响概率, 第四层给出放行判定

直觉上, 前三层像给写作者的指导, Layer 4 像出门前的门禁。可以用一个很小的符号系统表达这个边界:

$$y \sim p_{\theta}(\cdot \mid x, C_1, C_2(u, t), V_3), \quad D(y, R) \in \{\text{allow}, \text{flag}, \text{reject}\}.$$

- x : 用户消息与相关对话上下文。
- C_1 : Layer 1 的不可变约束, 例如 AI 身份披露、身体在场边界、禁用高风险词。
- $C_2(u, t)$: Layer 2 的用户 u 在时间 t 下的情境信息, 例如角色、压力、当前业务条件。
- V_3 : Layer 3 的语气示例、风格旋钮和编辑样本。
- p_{θ} : 参数为 θ 的语言模型生成分布; 它表达“更可能写出什么”。
- $\cdot$: 所有可能输出文本的占位符。
- y : 模型实际生成的草稿。
- R : 现实来源或允许列表, 例如日历、价格表、政策库、身份配置、禁用词表。
- $D(y, R)$: 生成后否决函数, 它读取草稿 y , 并根据现实来源 R 作出判定。
- `allow`: 允许发送。
- `flag`: 标记复核。

¹⁶ 视频画面时间区间: 00:15:02.59–00:16:32.59。

- reject: 拒绝发送。

确定性边界

提示词负责 prevention (预防), veto 负责 check (检查)。预防降低错误概率, 检查给出放行边界。若错误具体事实会让真人采取行动, 系统需要两者同时存在。

5.4 本章小结

Layer 4 的教学重点很直接: 模型写得像品牌, 并不等于内容可以交付。soft flag 处理可疑质量问题, hard reject 处理高成本具体事实, numbers guard 把日期、数字和承诺拉回到真实数据源。真正的边界来自输出后的许可判断: 模型可以被请求遵守, 系统必须决定是否放行。

6 多租户系统中的确定性边界与可改进之处

本章主张

多租户 AI 系统能共享一套架构, 前提是身份、条件和 veto 都有确定性边界。一个架构可以服务许多品牌声音; 每个租户的身份必须显式加载, 高风险输出必须经过同一个默认闸门。

讲者在收尾处把四层模式从“品牌语气”推进到系统架构问题。所谓 **multi-tenant (多租户)**, 指同一套代码同时服务多个场地、产品或业务表面。共享架构节省维护成本, 也带来一种隐蔽风险: 若品牌身份、上下文条件或输出检查依赖分散配置, 某个租户可能在用户面前说出另一个租户的声音。

6.1 身份配置必须显式失败

在 Bloom 的场景里, Layer 1 对每个租户都相同; Layer 2 和 Layer 3 会按场地加载不同条件与语气。这个设计让同一套根逻辑同时服务婚礼场地、Threadline 这类完全不同的产品, 也让配置缺失变得更危险。

One architecture. Different drivers.

```
// personality-builder.ts — the bug that taught the lesson

// brand-identity fields are NOT defaulted here.
// Defaulting silently shipped every venue as
// "Sage" / "sage@hawthornemanor.com" — a white-label leak.

export function requireAiName(config, venueId): string {
  const name = config?.ai_name?.trim()
  if (!name) {
    throw new Error(`ai_name missing for venue ${venueId}`)
  }
  return name
}
```

Identity must never have a default. A missing brand identity is a crash, not a fallback. Fail loud — the quiet failure is a venue speaking in a stranger's voice.

ISADORA MARTIN-DYE

STOP WRITING TONE INSTRUCTIONS. LAYER THEM.

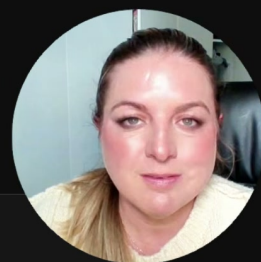


图 17: 多租户系统的身份风险: `requireAiName` 在缺少 `ai_name` 时抛错, 避免所有场地静默继承同一个品牌身份。¹⁷

讲者提到的 **white-label leak** (白标泄漏) 很具体: 品牌身份字段曾经静默默认, 导致每个场地都像 Sage 或 `sage@hawthornemanor.com` 那样发声。用户无法看到后台配置错误, 只会感觉这段话“不像这里的人”。信任就在这种微妙错位里流失。

多租户默认值的陷阱

身份字段适合采用 **fail loud (大声失败)**: 缺少品牌身份时让调用失败, 并把错误暴露给工程系统。静默 fallback 会把内部配置问题包装成外部品牌体验, 排查成本高, 伤害已经发生。

用 Google Maps 类比, 静默默认像每个司机都拿到同一个“家”的地址。路线规划器依旧能算路, 目的地已经错了。多租户 AI 的身份层也一样: 如果“谁在说话”没有确定, 后续语气再精致也会偏航。

6.2 请求与许可应由不同机制承担

四层模式最重要的系统分工是: Layer 1-3 属于 instruction (指令), Layer 4 属于 permission (许可)。讲者在幻灯片中把这句话压缩成: **“Instructions are a request. Permission is a decision.”**

¹⁷ 视频画面时间区间: 00:16:32.59–00:18:02.59。

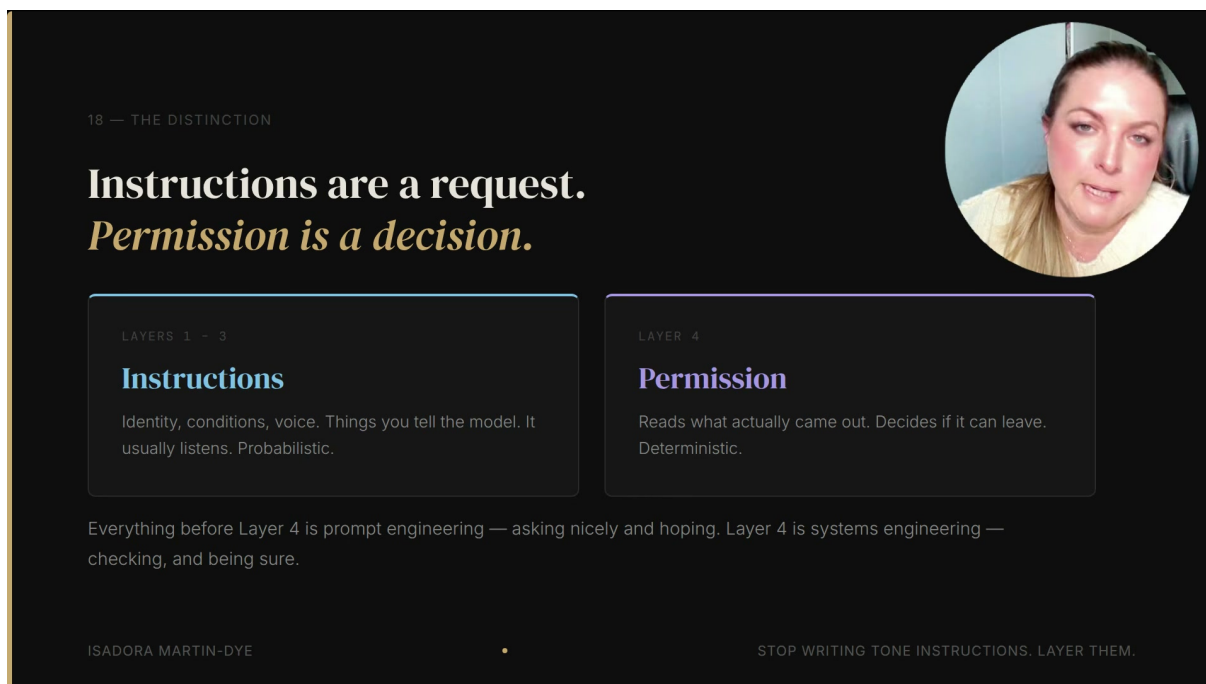


图 18: 前三层告诉模型身份、条件和语气；Layer 4 读取实际输出并决定能否离开业务系统。¹⁸

Prompt engineering 与 systems engineering

prompt engineering 通过文字让模型更可能遵循预期；**systems engineering** 通过组件边界、数据来源、失败模式和默认闸门控制实际行为。品牌语气属于前者，高成本事实放行属于后者。

这个分工防止一个提示词承担四个互相冲突的职责：不可违反、能读处境、有表达力、还能自我检查。模型擅长在上下文中生成自然语言；它对“哪一句话必须被拒绝”的判断需要外部现实来源与可追踪规则来兜底。

¹⁸ 视频画面时间区间：00:18:02.59–00:19:32.59。

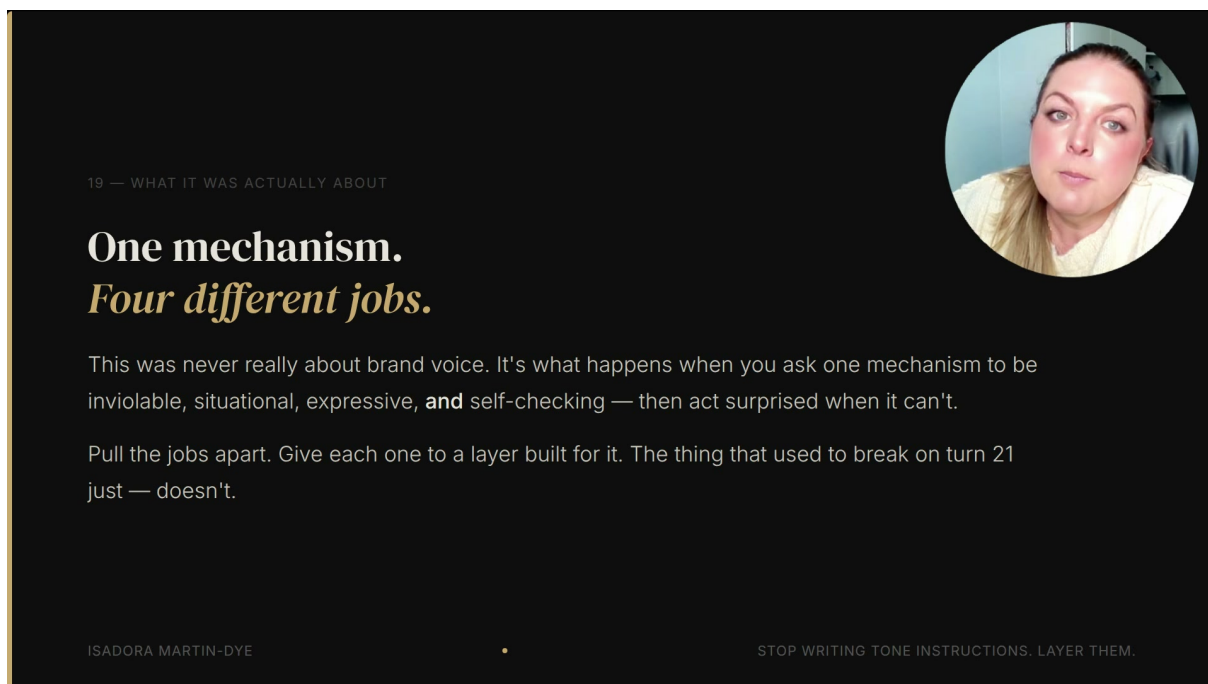


图 19: 一个机制被要求同时承担四项工作时，长尾失败会集中出现在第 21 轮：情境、身份、表达、检查需要分层处理。¹⁹

6.3 收尾建议一：把 veto 做成共享服务

讲者的第一条改进建议是把 veto 提升为独立共享服务。当前实现里，numbers guard 位于 heat narration 路径内部，honesty inspector 又是另一个独立函数。每新增一个业务表面，工程师都要记得手动接入检查，这等于把安全边界交给清单记忆。

更稳的架构是默认通过共享闸门：所有输出都进入同一个 veto service，由它统一执行软标记、硬拒绝、日志记录和复核路由。这样新增表面时无法无意绕过 Layer 4。对多租户系统而言，共享服务还可以统一处理不同租户的允许列表、禁用词、日历、价格表和政策源。

6.4 收尾建议二：集中条件解析

第二条建议是建立 **condition resolver**（条件解析器）。讲者说当前 latent mode detection 仍然部分依赖手工：heat narration 知道要加载情侣备注，briefing surface 知道某些备注不该加载。这些知识散在各个表面，长期会形成重复逻辑与遗漏风险。

集中条件解析器的职责可以概括为一句话：先看上下文，再决定路线。它读取用户身份、当前任务、业务表面、软上下文备注、实时约束，然后输出本轮应加载的 $C_2(u, t)$ 。这样 Layer 2 从“每个表面自己想起来”变成“一个中心组件显式裁决”。

条件解析器的工程收益

条件解析器让“谁为用户、正在发生什么、哪些条件可进入提示词”成为可测试接口。它还减少隐私泄露风险，因为软上下文可以被标注为“影响语气”或“可显式引用”，而非任由各个调用点自行处理。

¹⁹ 视频画面时间区间：00:18:02.59–00:19:32.59。

6.5 收尾建议三：正则拒绝与分类器的取舍

第三条建议是承认 **rejects / regex**（拒绝规则 / 正则规则）与 **classifier**（分类器）的取舍。正则和拒绝规则便宜、快速、确定；在覆盖到的模式上，它们要么匹配，要么不匹配。分类器可能抓到更多边缘情况，包括作者尚未写出规则的情况，同时也把概率性错误重新带回检查层。

因此选择取决于错误成本。若系统要拦截已知格式的日期、价格、邮箱、禁用词、身份字段，**regex/rejects** 很适合。若问题空间更开放，例如语义诱导、隐含承诺、复杂隐私泄露，分类器能扩大覆盖面。讲者当前选择 **determinism over coverage**（确定性优先于覆盖面），并明确说这是一个真实 **tradeoff**，需要团队按自己的风险曲线决定。

确定性检查也有边界

确定性 **veto** 的可靠性取决于数据源 *R*、允许列表完整度、规则维护和接入覆盖率。若日历同步错误、规则遗漏、某个新表面绕过共享闸门，系统仍会放出错误承诺。

6.6 信任是系统边界

讲者最后给出的产品判断非常尖锐：“**Users aren’t testing your brand voice. They’re trusting it.**” 用户不会像 QA 一样验证品牌语气，他们会把品牌声音当成服务承诺来理解。提示词总会在某个长尾时刻失败，关键在于这个失败被测试环境发现，还是被客户在真实对话里发现。

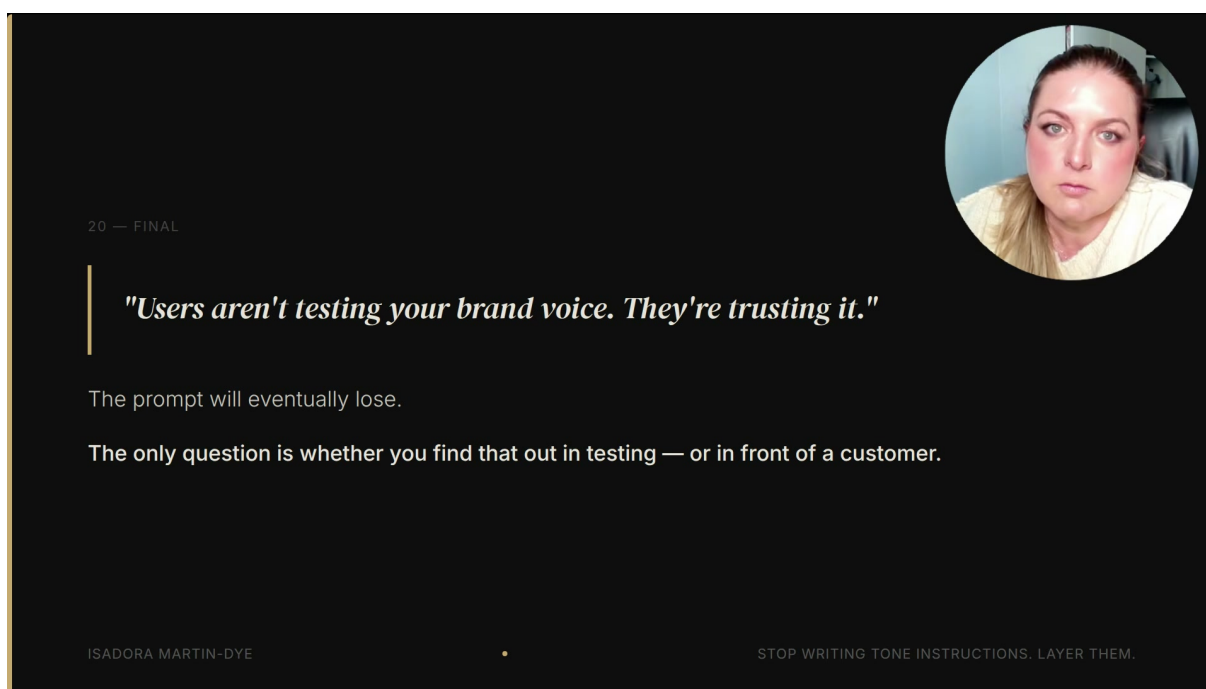


图 20: 收尾判断：用户信任品牌声音，提示词迟早会失败，系统应把失败暴露在测试和闸门中。²⁰

这也是本视频对构建者的最终要求：用提示词引导语气，用上下文决定路线，用不可变身份限定底线，用确定性 **veto** 守住高成本事实。品牌声音一旦代表业务对外说话，它就成为信任边界；信任边界需要工程化的失败模式、默认闸门和集中配置。

²⁰ 视频画面时间区间：00:18:02.59–00:19:32.59。

6.7 本章小结

多租户 AI 的成熟设计应把三个问题拉到台面上：身份缺失要 fail loud，条件选择要由 central condition resolver 管理，输出放行要进入 shared veto service。regex/rejects 适合已知低成本模式，classifier 适合扩展语义覆盖；两者的组合应围绕错误成本设计。最终标准很朴素：用户看到的是品牌承诺，系统必须在承诺离开业务之前完成检查。

7 总结与延伸

7.1 全片总论：品牌声音需要控制系统

这场演讲的中心结论可以压缩成一句话：当 AI 代理代表业务与用户建立关系时，品牌声音已经成为信任边界，必须用分层控制系统治理。单句 tone instruction 只能提高模型模仿品牌语气的概率；真正的工程结构要同时回答四个问题：哪些话永远不能说，当前用户处在什么情境，品牌应怎样表达，生成后的草稿能否放行。

最终压缩

Layer 1 给出不可变身份，Layer 2 读取实时处境，Layer 3 提供示例锚定语气，Layer 4 执行生成后许可。前三层让正确输出更可能出现，第四层检查实际输出能否离开业务系统。

这个框架的价值不只在婚礼场地。凡是 AI 代理会说出承诺、身份、事实、价格、日期、政策、情绪判断或安全敏感词的地方，都存在同一类风险：用户不会把品牌声音当实验样本来测试，他们会把它当成业务正在对自己说话。

7.2 讲者的收尾讨论：三个还应继续工程化的部位

讲者最后没有把四层模式包装成完美框架，而是明确指出三个仍需改进的工程点。第一，veto 应该成为共享服务。若每个业务表面都要手动接入 numbers guard 或 honesty inspector，未来某个新入口迟早会漏接。共享闸门让所有输出默认经过同一套软标记、硬拒绝、日志和复核路径。

第二，condition resolver 应该集中化。当前哪些表面加载情侣备注、哪些表面加载内部 briefing 条件，仍然部分散在各调用点。集中条件解析器可以把“此刻是谁、处在什么任务、哪些软上下文可用、哪些约束应进入提示”变成一个可测试接口。

第三，rejects/regex 与 classifier 的取舍要说清楚。正则拒绝规则便宜、快、确定，适合日期、价格、身份字段、禁用词这类已知模式；分类器可能覆盖更开放的语义风险，也会重新带来概率性误判。讲者当前选择 determinism over coverage，这个选择适合低成本、可枚举的错误面；更开放的风险面可能需要二者组合。

工程选择的直觉

确定性规则像门禁卡，识别范围有限，命中后结论清楚。分类器像安检人员，能识别更多可疑情境，也可能判断失误。高风险具体事实优先用门禁卡；开放语义风险可以增加安检人员，并保留复核路径。

7.3 给构建者的实践顺序

如果读者要把这套方法落到自己的 AI 产品里，可以按四步推进。第一步，列出不可变身份规则：AI 是否必须披露身份，哪些动作暗示真人身体，哪些词会被用户当成事实确认或承诺。第二步，建立情境信号表：角色、渠道、用户状态、软上下文、实时业务条件分别从哪里来，哪些能明示，哪些只能影响语气。

第三步，收集好回答样本：真实人工回复、编辑前后对比、短语偏好、语气刻度和品牌禁区。第四步，给低成本事实接入 veto：日期查日历，价格查报价源，政策查政策库，身份查租户配置，禁用词查显式列表。只有这四步连起来，品牌声音才从“好听”进入“可靠”。

最容易漏掉的未知风险

团队通常会主动写 tone guide，却很少主动写“品牌没有资格说什么”。高风险边界往往藏在普通词里：meet、hold、available、matched、confirmed、guaranteed、soon、always。它们在日常英文里自然，在业务关系里可能变成承诺。

7.4 本章小结

这份笔记最终想保留的判断很直接：提示词能引导模型，系统边界负责保护用户。Layer 1 到 Layer 3 是生成前的方向盘，Layer 4 是输出前的闸门。对于低成本语气偏差，概率性指令已经有价值；对于用户会据此行动的事实和承诺，确定性检查是必要的工程边界。

因此，“Stop Writing Tone Instructions. Layer Them.”的真正含义是：不要把品牌声音当成一句文案要求。它应被拆成身份、情境、风格和许可四个控制面。拆开之后，团队才能看见每个失败模式该由谁负责，也才能在第 21 轮对话到来之前，把风险拦在客户之外。